7- Implement different types of Joins: Implement various types of joins in SQL, including Inner Join, Outer Join, and Natural Join, with suitable examples.

8- Implement Grouping and Ordering: Study and implement the GROUP BY and HAVING clauses, ORDER BY clause.

9- Implement Procedure.(Basic procedure, procedure for Insert, Update and Delete)

10- Implement Triggers(validation before and after insert, delete and update)

queries.sql

```sql
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(30)
);
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(30),
    Salary INT,
    Dept_ID INT
);

INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ISE'),
(3, 'ECE'),
(4, 'ME');

INSERT INTO Employee VALUES
(101, 'Rohit', 30000, 1),
(102, 'Rahul', 25000, 2),
(103, 'Ramya', 35000, 1),
(104, 'Anu', 20000, NULL);

SELECT Employee.Emp_ID, Employee.Emp_Name,
       Department.Dept_Name
FROM Employee
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+--------+----------+-----------+
| Emp_ID | Emp_Name | Dept_Name |
+--------+----------+-----------+
|    101 | Rohit    | CSE       |
|    102 | Rahul    | ISE       |
|    103 | Ramya    | CSE       |
+--------+----------+-----------+
+----------+-----------+
| Emp_Name | Dept_Name |
+----------+-----------+
| Rohit    | CSE       |
| Rahul    | ISE       |
| Ramya    | CSE       |
| Anu      | NULL      |
+----------+-----------+
+----------+-----------+
| Emp_Name | Dept_Name |
+----------+-----------+
```

queries.sql

```sql
INNER JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;

SELECT Employee.Emp_Name, Department.Dept_Name
FROM Employee
LEFT OUTER JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;

SELECT Employee.Emp_Name, Department.Dept_Name
FROM Employee
RIGHT OUTER JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;

SELECT Employee.Emp_Name, Department.Dept_Name
FROM Employee
LEFT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID

UNION

SELECT Employee.Emp_Name, Department.Dept_Name
FROM Employee
RIGHT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;

SELECT Emp_ID, Emp_Name, Dept_Name
FROM Emplovee
RIGHT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;

SELECT Emp_ID, Emp_Name, Dept_Name
FROM Employee
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
| Ramya    | CSE       |
| Rohit    | CSE       |
| Rahul    | ISE       |
| NULL     | ECE       |
| NULL     | ME        |
+----------+-----------+
+----------+-----------+
| Emp_Name | Dept_Name |
+----------+-----------+
| Rohit    | CSE       |
| Rahul    | ISE       |
| Ramya    | CSE       |
| Anu      | NULL      |
| NULL     | ECE       |
| NULL     | ME        |
+----------+-----------+
+--------+----------+-----------+
| Emp_ID | Emp_Name | Dept_Name |
+--------+----------+-----------+
|    101 | Rohit    | CSE       |
|    102 | Rahul    | ISE       |
|    103 | Ramya    | CSE       |
```

queries.sql

```sql
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(30),
    Salary INT,
    Dept_ID INT
);

INSERT INTO Employee VALUES
(101, 'Rohit', 30000, 1),
(102, 'Rahul', 25000, 2),
(103, 'Ramya', 35000, 1),
(104, 'Anu', 20000, 2),
(105, 'Arun', 40000, 3);

SELECT Dept_ID, COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Dept_ID;

SELECT Dept_ID, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Dept_ID;

SELECT Dept_ID, COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Dept_ID
HAVING COUNT(*) > 1;
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+---------+-----------------+
| Dept_ID | Total_Employees |
+---------+-----------------+
|       1 |               2 |
|       2 |               2 |
|       3 |               1 |
+---------+-----------------+
+---------+-------------+
| Dept_ID | Total_Salary |
+---------+-------------+
|       1 |        65000 |
|       2 |        45000 |
|       3 |        40000 |
+---------+-------------+
+---------+-----------------+
| Dept_ID | Total_Employees |
+---------+-----------------+
|       1 |               2 |
```

```sql
SELECT *
FROM Employee
ORDER BY Salary ASC;

SELECT *
FROM Employee
ORDER BY Salary DESC;

SELECT Dept_ID, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Dept_ID
HAVING SUM(Salary) > 30000
ORDER BY Total_Salary DESC;
```

```
|       2 |                2 |
+---------+------------------+
```

```
+--------+----------+--------+---------+
| Emp_ID | Emp_Name | Salary | Dept_ID |
+--------+----------+--------+---------+
|    104 | Anu      |  20000 |       2 |
|    102 | Rahul    |  25000 |       2 |
|    101 | Rohit    |  30000 |       1 |
|    103 | Ramya    |  35000 |       1 |
|    105 | Arun     |  40000 |       3 |
+--------+----------+--------+---------+
```

```
+--------+----------+--------+---------+
| Emp_ID | Emp_Name | Salary | Dept_ID |
+--------+----------+--------+---------+
|    105 | Arun     |  40000 |       3 |
|    103 | Ramya    |  35000 |       1 |
|    101 | Rohit    |  30000 |       1 |
|    102 | Rahul    |  25000 |       2 |
|    104 | Anu      |  20000 |       2 |
+--------+----------+--------+---------+
```

```
+---------+--------------+
| Dept_ID | Total_Salary |
+---------+--------------+
|       1 |        65000 |
|       2 |        45000 |
|       3 |        40000 |
+---------+--------------+
```

queries.sql

```sql
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(30),
    Marks INT
);

INSERT INTO Student VALUES
(101, 'Rohit', 'CSE', 85),
(102, 'Rahul', 'ISE', 75),
(103, 'Ramya', 'CSE', 90);

DELIMITER //

CREATE PROCEDURE DisplayStudents()
BEGIN
    SELECT * FROM Student;
END //

DELIMITER ;

CALL DisplayStudents();

DELIMITER //

CREATE PROCEDURE InsertStudent(
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+------------+-------+------------+-------+
| Student_ID | Name  | Department | Marks |
+------------+-------+------------+-------+
|        101 | Rohit | CSE        |    85 |
|        102 | Rahul | ISE        |    75 |
|        103 | Ramya | CSE        |    90 |
+------------+-------+------------+-------+
```

```
+------------+-------+------------+-------+
| Student_ID | Name  | Department | Marks |
+------------+-------+------------+-------+
|        101 | Rohit | CSE        |    95 |
|        102 | Rahul | ISE        |    75 |
|        104 | Anu   | ECE        |    80 |
+------------+-------+------------+-------+
```

```sql
CREATE PROCEDURE InsertStudent(
    IN sid INT,
    IN sname VARCHAR(50),
    IN dept VARCHAR(30),
    IN smarks INT
)
BEGIN
    INSERT INTO Student
    VALUES (sid, sname, dept, smarks);
END //

DELIMITER ;

CALL InsertStudent(104, 'Anu', 'ECE', 80);

DELIMITER //

CREATE PROCEDURE UpdateStudent(
    IN sid INT,
    IN newmarks INT
)
BEGIN
    UPDATE Student
    SET Marks = newmarks
    WHERE Student_ID = sid;
END //
```

STDIN

Input for the program (Optional)

Output:

```
+------------+-------+------------+-------+
| Student_ID | Name  | Department | Marks |
+------------+-------+------------+-------+
|        101 | Rohit | CSE        |    85 |
|        102 | Rahul | ISE        |    75 |
|        103 | Ramya | CSE        |    90 |
+------------+-------+------------+-------+
+------------+-------+------------+-------+
| Student_ID | Name  | Department | Marks |
+------------+-------+------------+-------+
|        101 | Rohit | CSE        |    95 |
|        102 | Rahul | ISE        |    75 |
|        104 | Anu   | ECE        |    80 |
+------------+-------+------------+-------+
```

```sql
DELIMITER //

CREATE PROCEDURE DeleteStudent(IN sid INT)
BEGIN
    DELETE FROM Student
    WHERE Student_ID = sid;
END //

DELIMITER ;

CALL DeleteStudent(103);

SELECT * FROM Student;
```

queries.sql

```sql
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

CREATE TABLE Student_Log (
    Message VARCHAR(100)
);

DELIMITER //

CREATE TRIGGER BeforeStudentInsert
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    IF NEW.Marks < 0 OR NEW.Marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
    END IF;
END //

DELIMITER ;

DELIMITER //
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+------------+-------+-------+
| Student_ID | Name  | Marks |
+------------+-------+-------+
|        101 | Rohit |    95 |
+------------+-------+-------+
+-------------------------------+
| Message                       |
+-------------------------------+
| Student inserted successfully |
| Student updated successfully  |
+-------------------------------+
```

queries.sql

```
CREATE TRIGGER AfterStudentInsert
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES ('Student inserted successfully');
END //

DELIMITER ;

INSERT INTO Student VALUES (101, 'Rohit', 85);

DELIMITER //

CREATE TRIGGER BeforeStudentUpdate
BEFORE UPDATE ON Student
FOR EACH ROW
BEGIN
    IF NEW.Marks < 0 OR NEW.Marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
    END IF;
END //

DELIMITER ;

DELIMITER //
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+------------+-------+-------+
| Student_ID | Name  | Marks |
+------------+-------+-------+
|        101 | Rohit |    95 |
+------------+-------+-------+
+-------------------------------+
| Message                       |
+-------------------------------+
| Student inserted successfully |
| Student updated successfully  |
+-------------------------------+
```

queries.sql

```
CREATE TRIGGER AfterStudentUpdate
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES ('Student updated successfully');
END //

DELIMITER ;

UPDATE Student
SET Marks = 95
WHERE Student_ID = 101;

DELIMITER //

CREATE TRIGGER BeforeStudentDelete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
    IF OLD.Student_ID = 101 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete this student';
    END IF;
END //
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
+------------+-------+-------+
| Student_ID | Name  | Marks |
+------------+-------+-------+
|        101 | Rohit |    95 |
+------------+-------+-------+
+-------------------------------+
| Message                       |
+-------------------------------+
| Student inserted successfully |
| Student updated successfully  |
+-------------------------------+
```

```
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER AfterStudentDelete
AFTER DELETE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES ('Student deleted successfully');
END //

DELIMITER ;

SELECT * FROM Student;

SELECT * FROM Student_Log;
```

```
| Student_ID | Name  | Marks |
+------------+-------+-------+
|        101 | Rohit |    95 |
+------------+-------+-------+
+-------------------------------+
| Message                       |
+-------------------------------+
| Student inserted successfully |
| Student updated successfully  |
+-------------------------------+
```